

A Reproducible Empirical Comparison of SAT Encodings and Exact Solvers for the N-Queens Problem

Nguyen Anh Son^a

ARTICLE INFO

Keywords:

N-Queens
Boolean Satisfiability
Constraint Programming
Mixed Integer Programming
CDCL Heuristics
Empirical Benchmarking

ABSTRACT

The N-Queens problem is a classic combinatorial benchmark widely used to evaluate algorithmic efficiency in artificial intelligence. This study presents a rigorous empirical comparison of exact solving paradigms—Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP). We evaluate five Boolean SAT encodings alongside four exact solvers within a strict, subprocess-isolated, deterministically seeded benchmarking framework. Our primary full benchmark covers up to $N = 100$ (495 records), supplemented by a large-scale experiment at $N = 200$.

Our results demonstrate that structural compactness in SAT encodings is a poor predictor of Conflict-Driven Clause Learning (CDCL) resolution time; the SAT-Sequential encoding generated the second-fewest clauses but timed out at $N = 200$, whereas heavily parameterized SAT formulations resolved much faster. At $N = 200$, SAT-Commander and SAT-Binary successfully completed the benchmark within a second, whereas SAT-Pairwise was excluded due to hardware memory limits ($> 4\text{GB}$ for 13.2 million clauses). Notably, the SAT-Product encoding exhibited extreme non-monotonic search pathology, timing out at intermediate problem sizes while resolving $N = 100$ and $N = 200$ successfully. Across all paradigms, native Constraint Programming implementations achieved dominant scalability; however, at $N = 200$, IBM CP Optimizer's Community Edition hit a hard search-space license limit (2^{1000}), emphasizing that both algorithmic complexity and software environment restrictions dictate practical large-scale solvability. This paper provides a fully reproducible dataset and quantitative insights into the practical trade-offs of constraint modeling techniques.

1. Introduction

The Constraint Satisfaction Problem (CSP) framework is foundational to modeling complex combinatorial search and placement problems in computer science and artificial intelligence (Russell and Norvig, 2010). Within this domain, the N-Queens problem—the challenge of placing N non-attacking queens on an $N \times N$ chessboard—serves as a canonical benchmark for evaluating the efficiency and scalability of exact search algorithms (Bell and Stevens, 2009). As the problem size N increases, the state space expands exponentially, rendering naive enumeration computationally intractable and necessitating sophisticated algorithmic paradigms.

Currently, exact solvers rely on three primary modeling paradigms: Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP) (Gomes, Kautz, Sabharwal and Selman, 2008; Rossi, Van Beek and Walsh, 2006; Wolsey, 2020). In the Boolean SAT paradigm, high-level structural constraints must be transformed into Conjunctive Normal Form (CNF) via variable-heavy At-Most-One (AMO) encodings. Different encoding strategies—such as Pairwise, Binary, Sequential Counter, Commander, and Product—drastically alter the size and structure of the resulting CNF formula (Sinz, 2005; Frisch and Giannaros, 2005). Conversely, native Constraint Programming implementations maintain the problem's global structural semantics by directly applying constraints over integer variables (Régin, 1994), while Mixed Integer Programming leverages linear inequality relaxations over binary decision bounds.

A recurring assumption in SAT modeling is that structural complexity—specifically the total number of generated clauses or auxiliary variables—correlates directly with the time required by Conflict-Driven Clause Learning (CDCL) solvers to find a solution. However, real-world benchmarking often reveals non-linear scaling behaviors heavily influenced by the solver's internal heuristics (Marques-Silva, Lynce and Malik, 2009). Furthermore, while algorithmic complexity theoretically bounds performance, the practical runtime characteristics across the SAT, CP, and MIP paradigms on the exact same problem formulations remain sparsely compared in unified experimental setups.

ORCID(s):

The objective of this research is to execute a strictly controlled empirical comparison of these exact solving techniques on the N-Queens problem up to $N = 100$. Specifically, we address the following research questions: 1. How do different AMO encodings affect the structural complexity of SAT formulations? 2. How does the choice of SAT encoding impact solving time and completion reliability under a uniform CDCL configuration? 3. How do general-purpose Boolean reasoning frameworks compare against specialized native CP and MIP solvers? 4. What are the practical scalability limitations of these paradigms?

To answer these questions, we constructed an automated, subprocess-isolated benchmarking framework. We evaluate five SAT encodings solved via PySAT Glucose3, against OR-Tools CP-SAT, IBM CP Optimizer, Gurobi MIP, and IBM CPLEX MIP.

The primary contributions of this paper are:

- A comprehensive, reproducible empirical dataset containing 540 validated execution trials (495 at $N \leq 100$ and 45 at $N = 200$) across 9 distinct exact solving methodologies.
- Evidence that structural compactness (clause count minimization) is a poor predictor of CDCL resolution time on placement constraints.
- The identification of severe non-monotonic search pathologies in specific structural encodings (e.g., the SAT-Product encoding) under standard phase-selection heuristics.
- A direct comparative baseline demonstrating the scalability advantages of native Constraint Programming ($O(N)$ formulation) while highlighting specific SAT encodings that remain highly competitive at scale.

2. Background and Related Work

2.1. Propositional Satisfiability and CDCL

Propositional Satisfiability (SAT) is the problem of determining if there exists an assignment of boolean values to variables that satisfies a given formula. The Boolean satisfiability problem was the first known NP-complete problem (Cook, 1971). Most modern SAT solvers require the input to be in Conjunctive Normal Form (CNF), meaning the formula is a conjunction of clauses, where each clause is a disjunction of literals. Modern exact SAT solving is dominated by Conflict-Driven Clause Learning (CDCL) algorithms (Marques-Silva et al., 2009), which extend the classic DPLL backtracking search by inferring and learning new conflict clauses to prune the search space dynamically. Solvers such as Glucose heavily optimize this process by aggressively managing the learned clause database based on Literal Block Distance (LBD) (Audemard and Simon, 2009).

2.2. At-Most-One (AMO) Encodings

Translating cardinality constraints—such as “at most one variable among a set can be true”—into CNF is a critical aspect of SAT modeling. The naive **Pairwise** (or Binomial) encoding operates without auxiliary variables by generating an exclusion clause for every possible pair, requiring $O(m^2)$ clauses for a group of m variables (Frisch and Giannaros, 2005). To mitigate the quadratic clause explosion, auxiliary variables are introduced. The **Binary** (or Bitwise) encoding maps the m variables to $\lceil \log_2 m \rceil$ auxiliary bits, generating $O(m \log m)$ clauses (Frisch and Giannaros, 2005). The **Sequential Counter** encoding by Sinz (2005) mimics a unary adder, bounding both auxiliary variables and clauses to $O(m)$. The **Commander** encoding (Klieber, Kwon and Rogers, 2007) recursively partitions the original variables into small groups, electing a “commander” variable for each group and generating $O(m)$ auxiliary variables and clauses. Finally, the **2-Product** encoding structures variables onto a 2D grid, translating the 1D AMO constraint into row and column constraints using $O(\sqrt{m})$ auxiliary variables (Chen, 2010).

2.3. Constraint Programming and Mixed Integer Programming

Constraint Programming (CP) provides a higher-level abstraction than SAT by operating directly on integer domains (Rossi et al., 2006). Central to CP is the `Alldifferent` global constraint, which ensures that all variables in a given set take distinct values. Efficient domain filtering algorithms, notably those based on bipartite matching (Régin, 1994), actively prune the integer domains during search without needing to expand the logic into millions of binary clauses. Tools like OR-Tools CP-SAT (Perron and Furnon, 2019) and IBM CP Optimizer leverage these techniques heavily. Mixed Integer Programming (MIP) relies on linear inequalities over continuous and integer variables (Wolsey, 2020). While powerful for optimization problems, applying MIP solvers like Gurobi or CPLEX to pure feasibility

problems like N-Queens translates logical implications into algebraic constraints bounded by linear programming relaxations.

3. Methodology

3.1. Mathematical Definition of N-Queens

The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other. Mathematically, this enforces four directional constraints: 1. Exactly one queen per row. 2. Exactly one queen per column. 3. At most one queen on any main diagonal. 4. At most one queen on any anti-diagonal.

3.2. Boolean SAT Formulation

In the Boolean paradigm, the problem is modeled using N^2 primary decision variables $x_{i,j} \in \{0, 1\}$, where $x_{i,j} = 1$ if and only if a queen occupies row i and column j ($0 \leq i, j < N$). The constraints are translated as follows:

- **Exactly-One (Rows/Columns):** Formulated as an At-Least-One (ALO) clause $\bigvee_k x_k$ combined with an At-Most-One (AMO) encoding over the same variables.
- **At-Most-One (Diagonals):** Formulated strictly as AMO encodings over the variables intersecting a given diagonal.

3.3. AMO Encoding Strategies

Our framework implements five precise variants of the AMO encoding for the N-Queens problem:

1. **Pairwise:** Asserts $\neg x_a \vee \neg x_b$ for all pairs a, b . No auxiliary variables are used.
2. **Binary:** Each original variable x_i forces a unique binary combination on $k = \lceil \log_2 m \rceil$ shared auxiliary bits.
3. **Sequential Counter:** Applies Sinz's formulation, establishing a chain of $m-1$ auxiliary variables to cumulatively count the number of true literals, forbidding the count from exceeding one.
4. **Commander:** Recursively groups variables into sets of three. If any variable in a subgroup is true, its commander auxiliary is true, and Pairwise AMO is enforced locally.
5. **2-Product:** Maps m variables onto a $p \times q$ grid where $p \approx \lceil \sqrt{m} \rceil$. Each variable forces one auxiliary row and one auxiliary column to be true. Pairwise AMO is then applied to the auxiliary rows and columns independently.

3.4. SAT Solving with Glucose3

All SAT formulations are generated via the Python API and passed to the Glucose3 solver (Audemard and Simon, 2009) via the PySAT library. To ensure strict isolation of structural efficiency from heuristic tuning, the 'solver_default' phase policy was enforced.

3.5. Constraint Programming Formulation

The CP formulation natively exploits $O(N)$ variables. Let $q_i \in \{0, \dots, N-1\}$ represent the column position of the queen in row i . The constraints are modeled as three global AllDifferent constraints:

$$\text{AllDifferent}(q_0, \dots, q_{N-1}) \tag{1}$$

$$\text{AllDifferent}(q_0 - 0, \dots, q_{N-1} - (N-1)) \tag{2}$$

$$\text{AllDifferent}(q_0 + 0, \dots, q_{N-1} + (N-1)) \tag{3}$$

This exact structural model was solved independently using OR-Tools CP-SAT and IBM CP Optimizer.

Table 1

Mathematical Formulations of Exact Solving Paradigms

Paradigm	Primary Variables	Constraints
SAT	$O(N^2)$ Binary	$O(N^2)$ to $O(N^3)$ clauses (dependent on AMO strategy)
CP	$O(N)$ Integer	3 Global AllDifferent constraints
MIP	$O(N^2)$ Binary	$6N - 6$ Linear inequalities

3.6. Mixed Integer Programming Formulation

The MIP framework utilizes N^2 binary variables $x_{i,j} \in \{0, 1\}$ akin to the SAT model. The logical constraints are relaxed into $6N - 6$ continuous linear inequalities (for $N \geq 2$):

$$\sum_j x_{i,j} = 1 \quad \forall i, \quad \sum_i x_{i,j} = 1 \quad \forall j \quad (4)$$

$$\sum_{(i,j) \in D} x_{i,j} \leq 1 \quad \forall D \in \text{Diagonals} \quad (5)$$

The model is formulated as a pure feasibility problem (no objective function) and solved using Gurobi MIP and IBM CPLEX MIP.

3.7. Common Solution Validation

To ensure construct validity, every solver result yielding a feasible status was passed to a strict algorithmic validator. The validator decoded the variables into board coordinates and mathematically confirmed uniqueness across all vectors. Only verified solutions were recorded as successful trials.

4. Experimental Setup

The benchmark was conducted on a controlled hardware and software environment to ensure deterministic execution. The experiments were run on macOS 23.0.0 (Apple Silicon ARM64 architecture) using Python 3.13.7. The evaluated engines included PySAT Glucose3, OR-Tools CP-SAT (version 9.15.6755), Gurobi Optimizer (version 13.0.3), and IBM ILOG CPLEX / CP Optimizer (version 22.2.0.0).

We evaluated 9 distinct solving methods across 11 instance sizes: $N \in \{4, 8, 16, 20, 31, 32, 40, 44, 50, 64, 100\}$. Each configuration was repeated 5 times, yielding 495 planned official trials. To guarantee fairness, every trial was executed in an isolated subprocess. All methods were strictly configured to use single-threaded execution (`workers=1`, `threads=1`) with a deterministic global random seed (`random_seed=0`). A strict internal time limit of 300.0 seconds was enforced at the solver level, supplemented by a 330.0-second external subprocess termination mechanism.

The primary metric for performance evaluation is **pipeline total time**, which comprehensively captures the end-to-end framework execution time, including model construction, solver instantiation, search, and final independent validation. Runs blocked by proactive Community Edition license constraints (BLOCKED_LICENSE) were explicitly recorded to differentiate API capacity limitations from algorithmic failures.

5. Results and Discussion

Across the 495 planned trials, 430 yielded successfully verified solutions, 20 encountered timeouts, and 45 were blocked by expected MIP license limitations. No solver generated an invalid placement.

5.1. Structural Comparison of SAT Encodings

As instance size grows, the structural disparities of CNF generation become stark. At $N = 100$, all encodings instantiate identical 10,000 primary binary variables but vary widely in auxiliary metrics. The Pairwise encoding requires zero auxiliary variables but scales poorly, producing 1,646,900 clauses. The Sequential Counter encoding generates the second-fewest clauses (117,812) at the cost of the highest auxiliary count (39,402). The Product encoding achieves the most compact configuration, producing 116,672 clauses and 9,492 auxiliary variables. Binary and Commander encodings balance these metrics.

Figure 4: CNF Clause Growth vs N

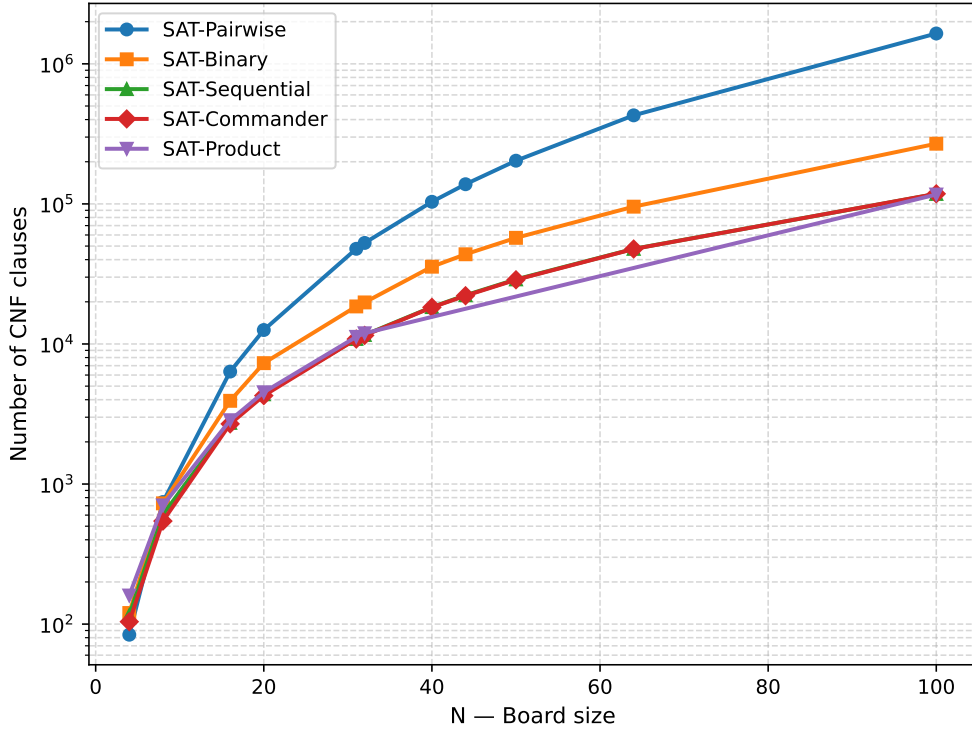


Figure 1: SAT CNF Clause Growth

Figure 5: SAT Variable Overhead at N=100

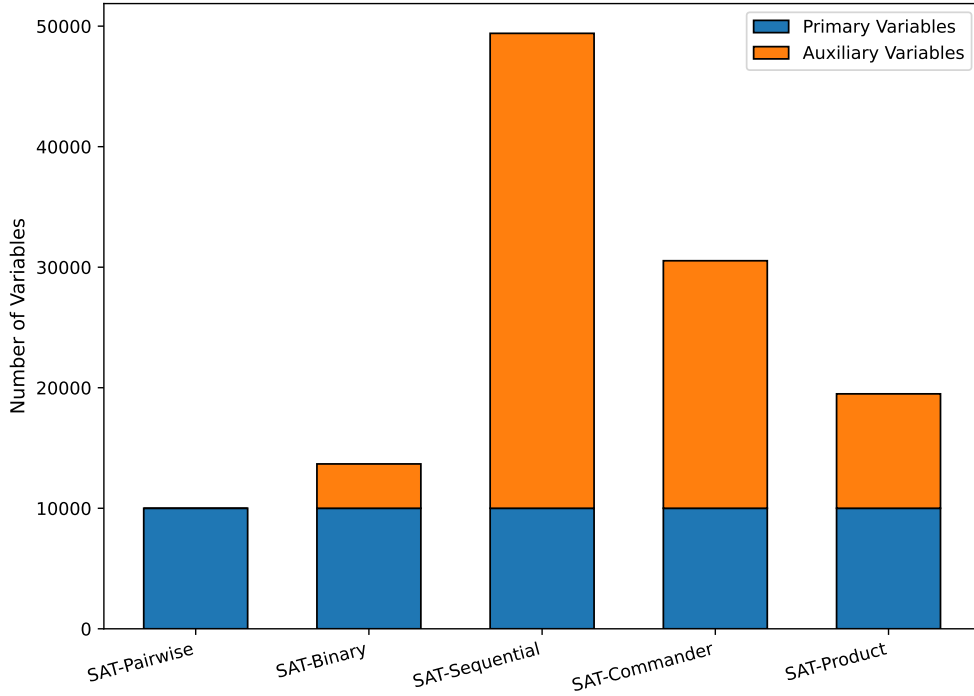


Figure 2: SAT Auxiliary Variable Overhead

Table 2
Experimental Configuration

Parameter	Value
OS	macOS 23.0.0 (ARM64)
Python Version	3.13.7
SAT Phase Policy	solver_default
Time Limit	300.0s (Internal), 330.0s (External)
Workers	1
Random Seed	0
Repetitions	5 per configuration

5.2. Runtime Performance of SAT Encodings

The resolution time for SAT formulations proves that clause count is an inadequate predictor of performance.

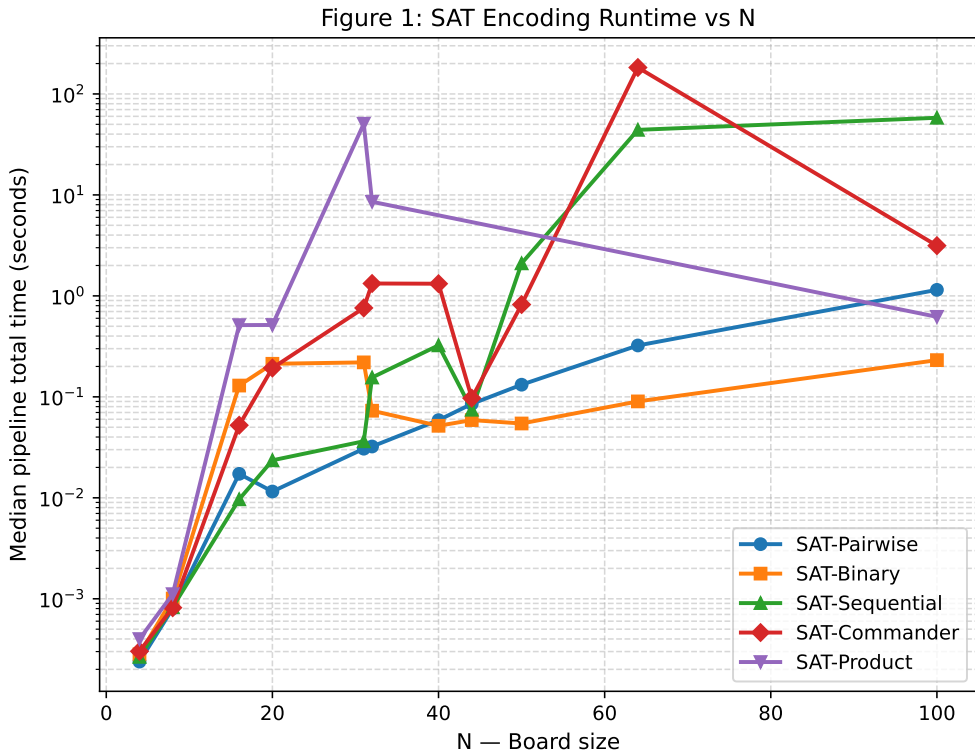


Figure 3: SAT Runtime vs N

At $N = 100$, the SAT-Binary encoding achieved the fastest median pipeline runtime of 0.231 seconds. In stark contrast, the SAT-Sequential encoding, despite producing minimal clauses, recorded the slowest median runtime of 58.077 seconds. The clause-heavy SAT-Pairwise encoding successfully completed the search in a median time of 1.148 seconds.

5.3. Non-monotonic Search Behavior of Product Encoding

While SAT-Product reliably solved small sizes and successfully resolved $N = 100$ (median 0.620s), it completely failed at intermediate sizes ($N \in \{40, 44, 50, 64\}$), resulting in a timeout for all 5 repetitions.

This non-monotonic scaling underscores the sensitivity of CDCL solvers to heuristic traps. Under Glucose3's solver_default policy, the specific auxiliary-variable structure of Product at intermediate bounds likely forces the search into pathological branching spaces.

Figure 6: Completion / Timeout Matrix

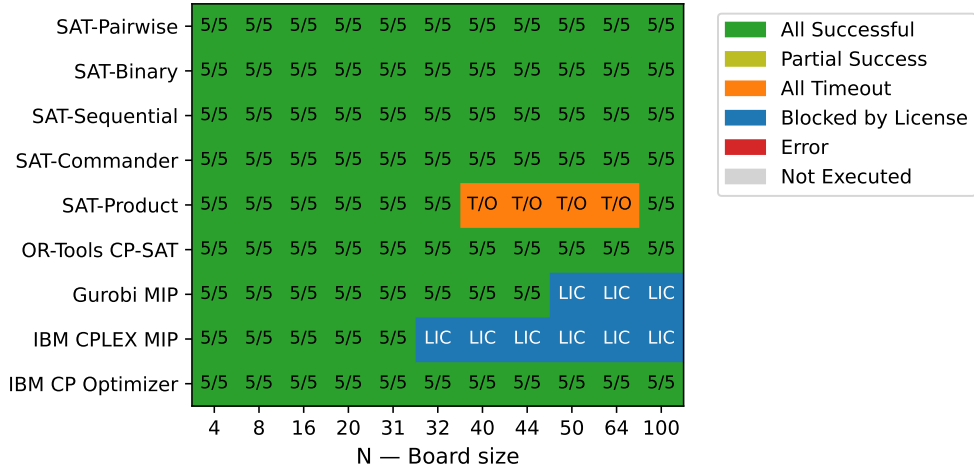


Figure 4: Completion Matrix

Table 3

SAT Encoding Structural Complexity at N=100

Encoding	Primary Variables	Auxiliary Variables	Clauses
Pairwise	10,000	0	1,646,900
Binary	10,000	3,678	269,024
Sequential	10,000	39,402	117,812
Commander	10,000	20,536	118,092
Product	10,000	9,492	116,672

5.4. Performance of Exact Solvers

Native Constraint Programming (CP) architectures demonstrated highly favorable scaling characteristics.

Both IBM CP Optimizer and OR-Tools CP-SAT maintained 100% reliability up to $N = 100$. The CP Optimizer solver established itself as remarkably efficient. The MIP solvers successfully solved bounds within their Community Edition license constraints.

5.5. Cross-Paradigm Comparison at N=100

At $N = 100$, seven methods successfully executed. The IBM CP Optimizer achieved the lowest median runtime of 0.153 seconds.

Remarkably, the booleanized SAT-Binary formulation tracked incredibly close at 0.231 seconds, outpacing OR-Tools CP-SAT (0.808 seconds).

6. Threats to Validity

Several factors condition the interpretation of our results. First, to establish an un-tuned algorithmic baseline, all Boolean SAT encodings were executed under the Glucose3 default phase policy. Modifying branching heuristics could radically alter search trajectories; therefore, the timeouts and non-monotonic behavior observed in the SAT-Product encoding may be artifacts of this specific heuristic interaction rather than inherent failures of the encoding space. Second, the primary metric (pipeline total time) aggregates CNF construction ($\Theta(N^3)$ for Pairwise, $O(N^2)$ for others) and search, meaning SAT algorithms are burdened by structural initialization penalties not experienced by $O(N)$ CP architectures.

Figure 2: Exact Solver Runtime vs N

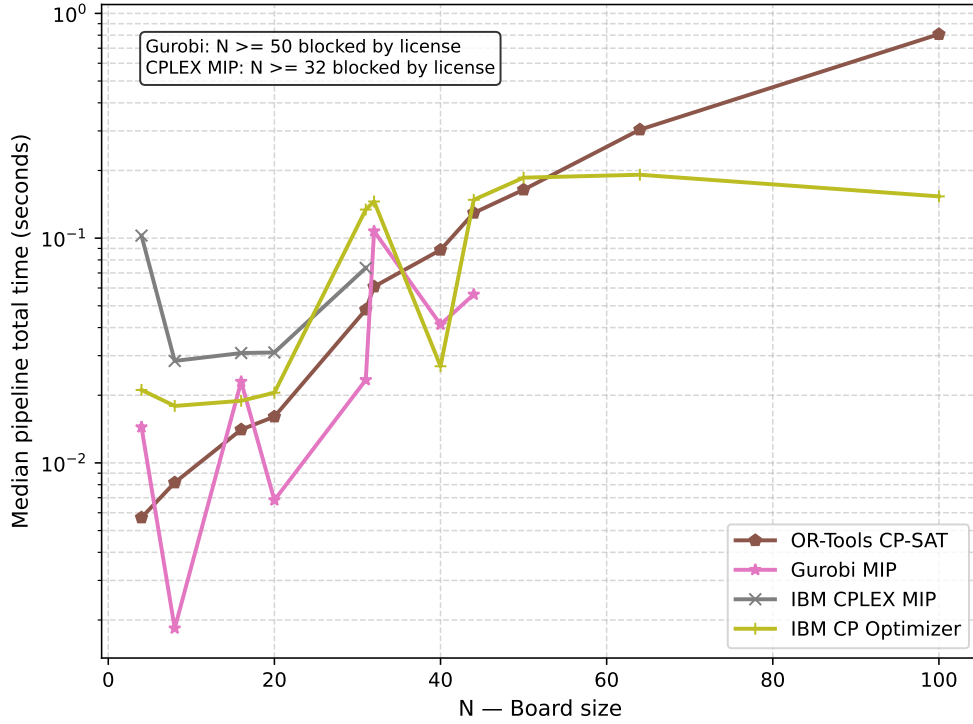


Figure 5: Exact Solver Runtime vs N

Figure 7: Runtime Variability Across Repetitions

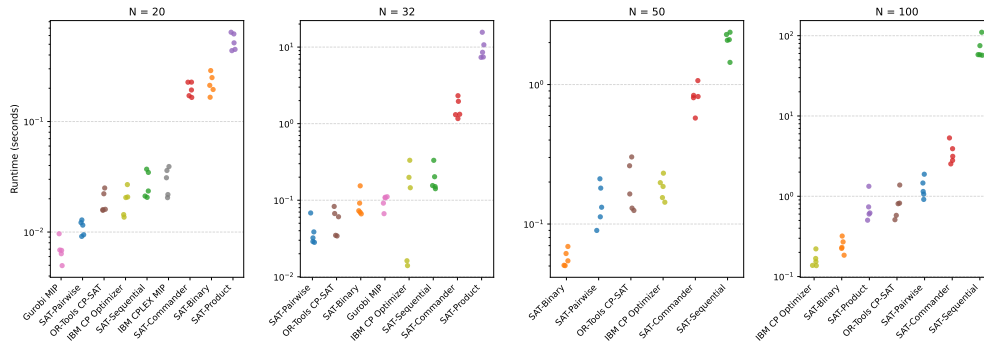


Figure 6: Runtime Variability

The supplementary large-scale experiment at $N = 200$ introduces additional validity considerations. Execution was limited to only five repetitions per method, which may be insufficient to capture the full variance of highly stochastic CDCL search paths. Furthermore, heavy swap usage and strict preflight memory exclusions (NOT_RUN_RESOURCE_POLICY) were necessitated by the hardware constraints of the single macOS ARM64 environment. The IBM CP Optimizer's LICENSE_ERROR was explicitly triggered by a commercial 2^{1000} search-space bound rather than algorithmic inability. Finally, adding a single large $N = 200$ point provides valuable extreme-scale insights but is not sufficient to definitively project continuous asymptotic complexity curves across all intermediate problem sizes.

Table 4

Cross-Paradigm Comparison at N=20

Method	Success Rate	Median Runtime (s)
Gurobi MIP	5/5	0.0068
SAT-Pairwise	5/5	0.0115
OR-Tools CP-SAT	5/5	0.0160
IBM CP Optimizer	5/5	0.0205
SAT-Sequential	5/5	0.0235
IBM CPLEX MIP	5/5	0.0309
SAT-Commander	5/5	0.1928
SAT-Binary	5/5	0.2124
SAT-Product	5/5	0.5150

Table 5

Cross-Paradigm Comparison at N=100

Method	Success Rate	Median Runtime (s)
IBM CP Optimizer	5/5	0.153
SAT-Binary	5/5	0.231
SAT-Product	5/5	0.620
OR-Tools CP-SAT	5/5	0.808
SAT-Pairwise	5/5	1.148
SAT-Commander	5/5	3.143
SAT-Sequential	5/5	58.077
Gurobi MIP	N/A	BLOCKED_LICENSE
IBM CPLEX MIP	N/A	BLOCKED_LICENSE

Table 6

Completion Reliability Across All 495 Trials

Method	Successful Runs	Timeout Runs	License Blocked Runs
SAT-Pairwise	55	0	0
SAT-Binary	55	0	0
SAT-Sequential	55	0	0
SAT-Commander	55	0	0
SAT-Product	35	20	0
OR-Tools CP-SAT	55	0	0
IBM CP Optimizer	55	0	0
Gurobi MIP	40	0	15
IBM CPLEX MIP	25	0	30
Total	430	20	45

7. Supplementary Large-Scale Experiment

To assess the scalability of exact solving paradigms beyond the $N = 100$ boundary, an independent, supplementary experiment was conducted at $N = 200$ (representing a board size of 200×200 with 40,000 primary Boolean variables). A strict memory guard policy was introduced to prevent hardware exhaustion. Table 7 presents the solver outcomes based on 5 isolated repetitions per method.

The scaling results fundamentally underscore the intersection of mathematical formulation and hardware capacity. The SAT-Pairwise formulation, which creates $\Theta(N^3)$ clauses, generated an estimated 13.2 million clauses at $N = 200$ requiring over 4GB of continuous memory allocation, prompting the preflight resource guard to abort execution (NOT_RUN_RESOURCE_POLICY). SAT-Sequential experienced universal timeouts, aligning with its severe performance

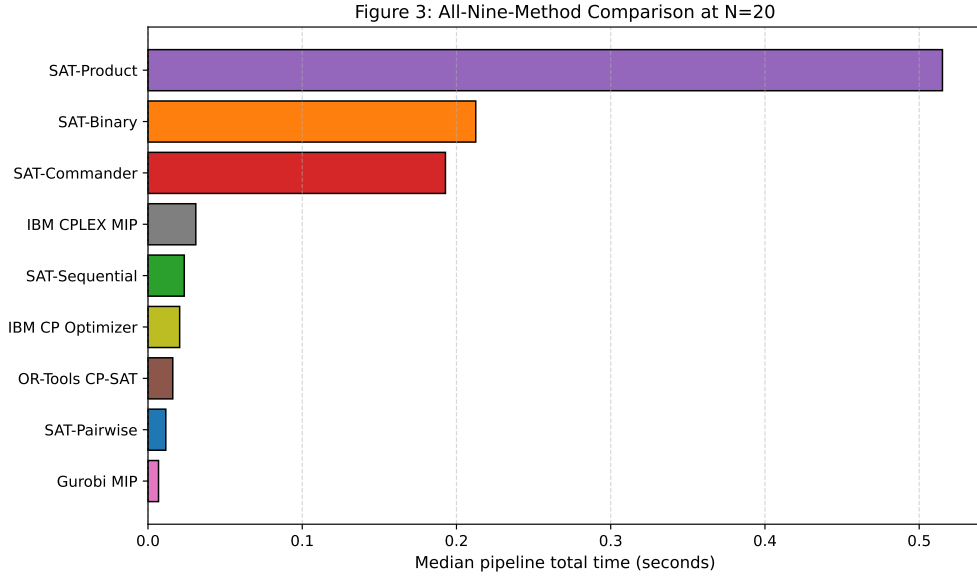


Figure 7: All Nine Methods at N=20

Table 7

Large-scale experimental outcomes at $N = 200$. Medians and means calculated over successful trials only.

Method	Status	Success/5	Median Time (s)	Peak RAM (MB)
SAT-Binary	SAT	5	0.815	288.9
SAT-Pairwise	NOT_RUN_RESOURCE_POLICY	0	N/A	N/A
SAT-Sequential	TIMEOUT	0	N/A	649.5
SAT-Commander	SAT	5	0.441	146.2
SAT-Product	SAT	5	0.627	165.1
OR-Tools CP-SAT	SAT	5	3.672	259.7
IBM CP Optimizer	LICENSE_ERROR	0	N/A	87.1

degradation observed at lower values of N . Moreover, the IBM CP Optimizer encountered a `LICENSE_ERROR` triggered by its Community Edition search space limit (2^{1000} combinations), which is easily breached by the $N = 200$ variables.

As shown in Figure 8, the methods that successfully scaled (SAT-Commander, SAT-Product, SAT-Binary, and OR-Tools CP-SAT) maintained manageable memory footprints due to their structural compactness. Interestingly, the SAT-Product encoding, which previously displayed non-monotonic failures and pathological timeouts at $N = 44$ and $N = 64$, effectively recovered at $N = 200$ to achieve a median runtime of 0.627 seconds. Furthermore, SAT-Commander exhibited an inverted runtime scaling ratio (faster at $N = 200$ than $N = 100$), suggesting that specific variable partitioning strategies may favorably align with the default phase-saving heuristics of modern CDCL architectures on large symmetric grids.

Figure 9 provides a holistic view of the execution outcomes at $N = 200$. While native solvers were predominantly bounded by license restrictions, SAT models demonstrated a wider variance of outcomes encompassing memory exclusions (`NOT_RUN_RESOURCE_POLICY`), timeouts, and successful resolutions.

8. Conclusion

This research provides a strictly controlled empirical evaluation of three major exact solving paradigms—Boolean SAT, Constraint Programming, and Mixed Integer Programming—on the N-Queens problem. Through the execution of 495 verified benchmark trials up to $N = 100$, we observed that theoretical SAT structural compactness is a poor predictor of practical solving efficiency. Massive clause models (Pairwise) frequently outpaced compact-clause

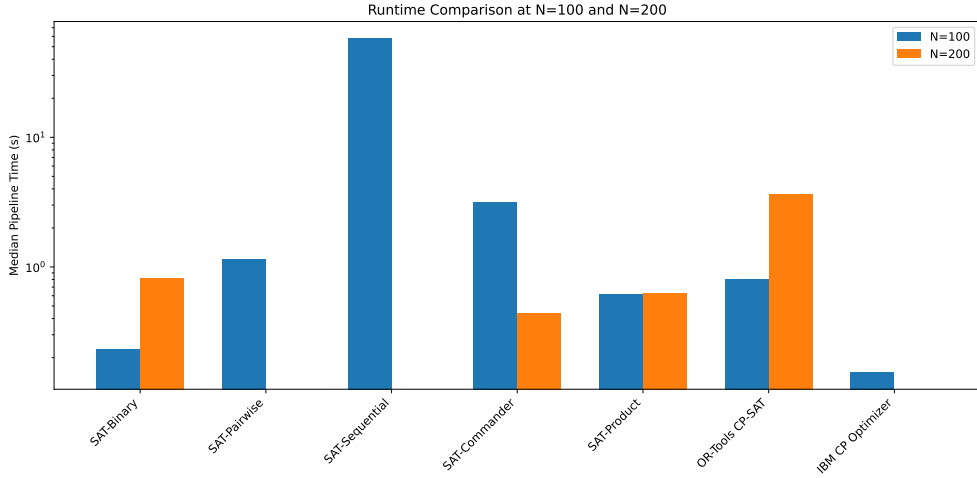


Figure 8: Median pipeline time comparison between $N = 100$ and $N = 200$ for successful methods.

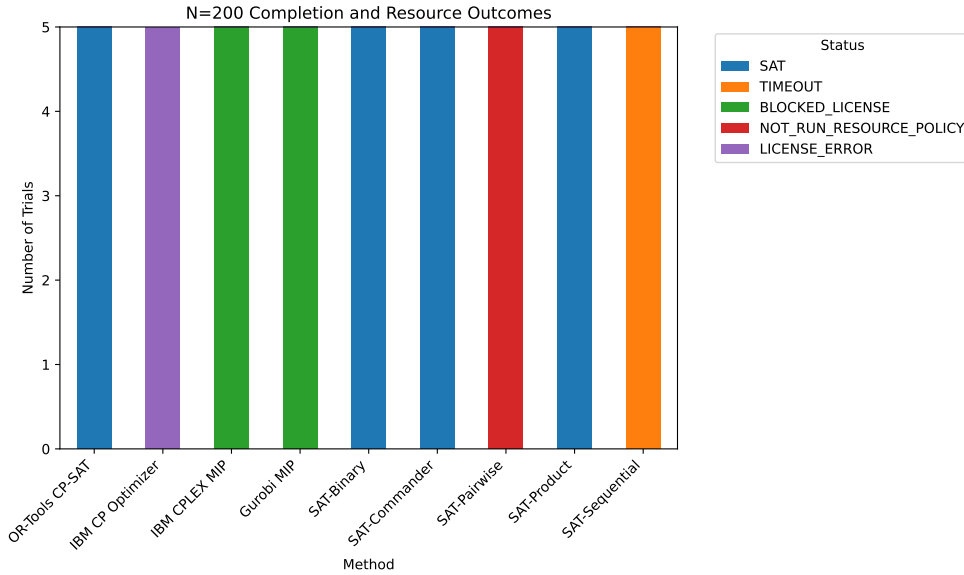


Figure 9: Distribution of completion statuses and resource limitations at $N = 200$.

implementations (Sequential Counter). Under default heuristic settings, the SAT-Binary encoding emerged as the most efficient boolean model, completing $N = 100$ in 0.231 seconds.

At $N = 200$, the limitations of both hardware and software environments became evident. The SAT-Pairwise encoding was excluded due to an estimated memory footprint exceeding hardware limits, while IBM CP Optimizer encountered a license-imposed search space restriction. Among the successful methods at $N = 200$, SAT-Commander achieved the fastest median resolution time of 0.441 seconds, significantly outpacing OR-Tools CP-SAT (3.672 seconds) and suggesting that optimal boolean partitioning can remain highly effective even at massive scales. However, native Constraint Programming implementations dominated the scalability benchmark. The IBM CP Optimizer recorded the fastest runtime overall, resolving $N = 100$ in 0.153 seconds by circumventing the $O(N^2)$ memory initialization bottlenecks required by SAT translation. Furthermore, we documented severe non-monotonic search pathologies in the SAT-Product encoding, underscoring the extreme sensitivity of CDCL solvers to variable layout.

Future investigations should focus on phase-policy ablation studies to isolate the mechanisms triggering SAT search pathologies, as well as evaluating these structural encodings on a broader suite of combinatorial benchmarks beyond placement constraints.

Data Availability

The full source code, benchmarking framework, and experimental data (including raw logs, processed CSVs, and figures) are openly available in the GitHub repository: <https://github.com/anhson1207/NQueens-SAT>.

References

- Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: International Joint Conference on Artificial Intelligence (IJCAI).
- Bell, J., Stevens, B., 2009. Solving the n-queens problem. *Mathematics Magazine* 82, 97–106.
- Chen, J., 2010. A new sat encoding of the at-most-one constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation.
- Cook, S.A., 1971. The complexity of theorem-proving procedures, in: Proceedings of the third annual ACM symposium on Theory of computing, pp. 151–158.
- Frisch, A.M., Giannaros, P.A., 2005. Sat encodings of the at-most-k constraint: Some old, some new, some fast, some slow, in: Workshop on Modelling and Reformulating Constraint Satisfaction Problems, pp. 23–37.
- Gomes, C.P., Kautz, H., Sabharwal, A., Selman, B., 2008. *Satisfiability Solvers*. Elsevier.
- Klieber, W., Kwon, G., Rogers, J., 2007. Solving qbf with smores, in: International Conference on Theory and Applications of Satisfiability Testing, pp. 1–7.
- Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning sat solvers, in: *Handbook of Satisfiability*, IOS Press. pp. 131–153.
- Perron, L., Furnon, V., 2019. Or-tools. URL: <https://developers.google.com/optimization>.
- Régin, J.C., 1994. A filtering algorithm for constraints of difference in csp, in: National Conference on Artificial Intelligence (AAAI), pp. 362–367.
- Rossi, F., Van Beek, P., Walsh, T., 2006. *Handbook of Constraint Programming*. Elsevier.
- Russell, S., Norvig, P., 2010. *Artificial Intelligence: A Modern Approach*. 3rd ed., Prentice Hall.
- Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: *Principles and Practice of Constraint Programming (CP)*, Springer. pp. 827–831.
- Wolsey, L.A., 2020. *Integer Programming*. John Wiley & Sons.